Reinforcement Learning

Second Assignment

Federica Lavini

Matricola: 1941986

Contents

1 Theory **3**

1.1 Q-learning and SARSA update . . . 3

1.1.1 Q-learning . . . 3

1.1.2 SARSA algorithm . . . 4

1.1.3 Conclusion . . . 4

1.2 N-step proof . . . 5

2 Practice **7**

2.1 Taxi Gymnasium: SARSA-λ algorithm . . . 7

2.1.1 Description of SARSA(λ) . . . 7

2.1.2 Implementation . . . 7

2.1.3 Analysis of Training Results . . . 8

2.2 Mountain Car: Value function approximation via Q-learning TD(λ) with RBF . . . 9

2.2.1 Description of Q-learning TD(λ) . . . 9

2.2.2 Implementation . . . 10

2.2.3 Analysis of Training Results and Evaluation Results . . . 10

3 Collaborations **12**

1 Theory

1.1 Q-learning and SARSA update

For the first theory exercise I need to compute the updates of the Q-learning and the SARSA algorithms

1.1.1 Q-learning

In this section I'll present the Q-learning algorithm and I'll apply it to the given exercise. Here is the Q-learning algorithm:

Algorithm 1 Q-Learning

1: Initialize $Q(s, a)$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, arbitrarily, except that $Q(\text{terminal-state}, \cdot) = 0$
2: **for** each episode **do**
3: Initialize S
4: **repeat**
5: Choose A from S using policy derived from Q (e.g., ε-greedy)
6: Take action A, observe R, S'
7: $Q(S, A) \leftarrow Q(S, A) + \alpha\,[R + \gamma \max_a Q(S', a) - Q(S, A)]$
8: $S \leftarrow S'$
9: **until** S is terminal
10: **end for**

The key element here is the update equation, line 7 of the algorithm.

So, given the following problem:

- **Q-table:** $\begin{bmatrix} Q(1,1) = 2 & Q(1,2) = 1 \\ Q(2,1) = 5 & Q(2,2) = 3 \end{bmatrix}$
- **Parameters:** $\alpha = 0.3$, $\gamma = 0.5$
- **Experience:** $(s, a, r, s') = (1, 1, 4, 2)$
- **SARSA-specific:** $a' = \pi_\epsilon(s') = 1$ (used only for SARSA)

To apply the Q-learning algorithm, first I need to compute $\max Q(s', a)$. We are looking for the maximum value in the next state, this represents the best possible action:

$$\begin{aligned} Q(2,1) &= 5 \\ Q(2,2) &= 3 \\ \max Q(s', a) &= \max(5, 3) = 5 \end{aligned}$$

Then, the target Q-learning is calculated:

$$\begin{aligned} \text{Target} &= r + \gamma \cdot \max Q(s', a) \\ &= 4 + 0.5 \cdot 5 \\ &= 4 + 2.5 = 6.5 \end{aligned}$$

and the TD error is calculated as well:

$$\begin{aligned} \text{Error TD} &= \text{Target} - Q(s, a) \\ &= 6.5 - 2 = 4.5 \end{aligned}$$

this represents the difference between the expectation and the actual reality.

Finally, I can apply the algorithm:

$$\begin{aligned} Q(s, a)_{\text{new}} &= Q(s, a) + \alpha \cdot \text{Error TD} \\ Q(1, 1)_{\text{new}} &= 2 + 0.3 \cdot 4.5 \\ &= 2 + 1.35 = 3.35 \end{aligned}$$

In the end, only $Q(1, 1)$ is updated:

$$Q_{\text{updated}} = \begin{bmatrix} 3.35 & 1 \\ 5 & 3 \end{bmatrix}$$

1.1.2 SARSA algorithm

The same problem will be approached using the SARSA algorithm. This section will first present the algorithm and then the solution of the exercise.

Algorithm 2 SARSA

1: Initialize $Q(s, a)$, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, arbitrarily, except that $Q(\text{terminal-state}, \cdot) = 0$
2: **for** each episode **do**
3: Initialize S
4: Choose A from S using policy derived from Q (e.g., ε-greedy)
5: **repeat**
6: Take action A, observe R, S'
7: Choose A' from S' using policy derived from Q (e.g., ε-greedy)
8: $Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma Q(S', A') - Q(S, A)\right]$
9: $S \leftarrow S'$
10: $A \leftarrow A'$
11: **until** S is terminal
12: **end for**

The first step to solve the problem is to identify the current Q value, where $(s, a) = (1, 1)$:

$$Q(1, 1) = 2$$

and the Q value of the next step, $Q(s', a')$ where $s' = 2$ e $a' = 1$:

$$Q(2, 1) = 5$$

Then, it's possible to apply the update formula, line 8:

$$Q(1, 1) \leftarrow 2 + 0.3\left[4 + 0.5 \times 5 - 2\right]$$

The new value for $Q(1, 1)$ after the SARSA update is:

$$3.35$$

and the new Q-table is:

$$Q_{\text{updated}} = \begin{bmatrix} 3.35 & 1 \\ 5 & 3 \end{bmatrix}$$

1.1.3 Conclusion

The computational results demonstrate that, for this specific experience, both the Q-learning and SARSA algorithms produced an identical update for the Q-table.

1.2 N-step proof

I want to prove:

$$G_{t:t+n} - V_{t+n-1}(S_t) = \underbrace{\sum_{k=t}^{t+n-1} \gamma^{k-t}\,\delta_k}_{\text{call it } S} ,$$

where

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$$

and the TD error is

$$\delta_k = R_{k+1} + \gamma V_{t+n-1}(S_{k+1}) - V_{t+n-1}(S_k).$$

Assumption I can write $V()$ instead of $V_{t+n-1}()$ because the value estimates do not change between steps.

Expand the S side of the equation

$$\sum_{k=t}^{t+n-1} \gamma^{k-t}\,\delta_k = \sum_{k=t}^{t+n-1} \gamma^{k-t}\Big(R_{k+1} + \gamma V(S_{k+1}) - V(S_k)\Big)$$

with the insertion the definition of δ_k. Then it's possible to obtain three separate sums:

$$= \underbrace{\sum_{k=t}^{t+n-1} \gamma^{k-t} R_{k+1}}_{\text{call this } REW} + \underbrace{\sum_{k=t}^{t+n-1} \gamma^{k-t+1} V(S_{k+1})}_{\text{call this } NEXT} - \underbrace{\sum_{k=t}^{t+n-1} \gamma^{k-t} V(S_k)}_{\text{call this } CURR} .$$

Expand the following terms: REW, NEXT, CURR

$$REW = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n}. \tag{1}$$

This sequence is the rewards that appears in $G_{t:t+n}$. To simplify the next steps: $j = k+1$, $k = t$, $j = t+1$, $k = t+n-1$ and $j = t+n$.

$$NEXT = \sum_{j=t+1}^{t+n} \gamma^{j-t} V(S_j).$$

$$CURR = \sum_{j=t}^{t+n-1} \gamma^{j-t} V(S_j).$$

Compute NEXT-CURR

$$NEXT = \gamma^1 V(S_{t+1}) + \gamma^2 V(S_{t+2}) + \cdots + \gamma^{n-1} V(S_{t+n-1}) + \gamma^n V(S_{t+n}).$$

$$CURR = 1 \cdot V(S_t) + \gamma^1 V(S_{t+1}) + \gamma^2 V(S_{t+2}) + \cdots + \gamma^{n-1} V(S_{t+n-1}).$$

It's possible to notice that all interior terms cancel, except for the first term of $CURR$ and the last term of $NEXT$.

So:

$$NEXT - CURR = \gamma^n V(S_{t+n}) - V(S_t). \tag{2}$$

Combination of the three terms Now it's possible to combine the two equations: 1 and 2.

$$REW + (NEXT - CURR) = \left(R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n}\right) + \gamma^n V(S_{t+n}) - V(S_t).$$

As the equation shows, $G_{t:t+n}$ is precisely the sum of the $n - step$ reward and the discounted value of the state at time $t + n$:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

Therefore:

$$REW + (NEXT - CURR) = G_{t:t+n} - V(S_t).$$

Final Result

$$G_{t:t+n} - V_{t+n-1}(S_t) = \sum_{k=t}^{t+n-1} \gamma^{k-t}\, \delta_k$$

2 Practice

2.1 Taxi Gymnasium: SARSA-λ algorithm

The main goal of this exercise is to implement SARSA-λ algorithm in a discrete environment like *Grid-World*. The objective is to build the Q-table, propagate the TD error with eligibility traces, and when there are meaningful rewards reduce exploration.

2.1.1 Description of SARSA(λ)

The keys ideas of the algorithm are:

- **On-policy TD control**: for selecting actions and updating the value faction the algorithm uses the same policy, usually the ε-greedy.
- **Temporal-Difference (TD) error**: updates are based on the TD error:

$$\delta = r + \gamma Q(s', a') - Q(s, a).$$

- **Eligibility traces**: the algorithm maintains a temporary memory of the recent state–action visits and the current trace is incremented as

$$E(s, a) \leftarrow E(s, a) + 1,$$

  and all traces decay proportionally to

$$E \leftarrow \gamma\lambda E.$$

- **Backward credit assignment**: The TD error is distributed across all previously visited state–action pairs, weighted by their eligibility traces.
- **Interpolation between SARSA and Monte Carlo**:
  - when $\lambda = 0$ the algorithm becomes standard one-step SARSA,
  - when $\lambda \to 1$, it behaves more like Monte Carlo learning.

  SARSA(λ) mixes short-term and long-term learning.
- **Improved learning speed**: eligibility traces help faster propagation of information, which makes learning quicker especially in environments where rewards are delayed or rare.

2.1.2 Implementation

For the implementation part it is asked to write the ε-greedy policy in the `epsilon_greedy_action(env, Q, state, epsilon)` and update the Q-table and the eligibility.

The first function returns an action according to the ε-greedy rule: with probability ε a random action is sampled (exploration), otherwise the greedy action is selected:

$$a = \arg\max_{a'} Q(s, a').$$

This balances exploiting and discovering.

For the SARSAλ learning loop, the Q-table and the eligibility traces E are initialized to zero:

$$Q(s, a) = 0, \qquad E(s, a) = 0 \quad \forall s, a.$$

Eligibility traces store how recently each state–action pair was visited, and determine how strongly each pair is updated when a TD error occurs. At the beginning of each episode, the eligibility traces are reset:

$$E[:] = 0.$$

This is necessary because traces only accumulate information during a single episode to propagate the TD error backward. Resetting them inside the `while not done` loop would erase all accumulated traces at every step, nullifying the benefits of SARSA(λ) and reducing it to standard one-step SARSA.

Then, during each episode, the agent interacts with the environment:

1. selects the action using the ε-greedy policy,
2. performs the action and observes (s', r, a'),
3. computes the TD error
$$\delta = r + \gamma Q(s', a') - Q(s, a),$$
4. updates the eligibility trace of the visited pair:
$$E(s, a) \leftarrow E(s, a) + 1,$$
5. updates the Q-values using all accumulated traces:
$$Q \leftarrow Q + \alpha\, \delta\, E,$$
6. decays all traces:
$$E \leftarrow \gamma \lambda E.$$

2.1.3 Analysis of Training Results

The SARSA(λ) implementation was executed multiple times with identical hyperparameters

$$\alpha = 0.2, \qquad \gamma = 0.99, \qquad \lambda = 0.9, \qquad \varepsilon_0 = 1.0,$$

over 10,000 training episodes and overall behavior remained consistent.

The environment introduces a high level of randomness: the agent starts from a random state, the passenger and destination locations are randomized and rewards are sparse. Across the runs, this value ranged from episode 2 to episode 108, with the majority of cases between episodes 5 and 30. This behaviour is expected, since a long sequence of correct actions is required before achieving the pickup and drop-off rewards.

In the implementation, the exploration rate decays only after the first positive reward is observed:

$$\text{if received_first_reward:} \quad \varepsilon \leftarrow 0.99\, \varepsilon.$$

This design choice has a strong impact:

- If the first reward is discovered early, the ε-decay begins sooner, leading to faster stabilization of the policy and typically higher evaluation scores.
- If the first reward is found late, the agent remains fully exploratory for a large number of episodes, delaying learning and slightly reducing asymptotic performance.

This explains the variability observed in the mean evaluation rewards, which ranged between 5.4 and 7.3 across runs. Eligibility traces clearly accelerated learning: once a rewarding trajectory was identified, the TD error was rapidly propagated backward across many visited state–action pairs, strengthening the entire sequence and improving convergence speed.

Differences in training outcomes arise from:

- **Random initial states**
- **Exploratory randomness**
- **Dependence on ε-decay trigger**
- **Reward structure of Taxi-v3**

Discussion and Conclusions Several important observations emerged from the training behaviour:

- **Eligibility traces accelerate learning**. They propagate the TD error across many steps, reinforcing entire successful action sequences.
- **Delayed ε-decay prevents premature exploitation**. This ensures sufficient exploration in the large state space before becoming greedy.
- **High initial exploration is crucial**. Sparse rewards require extensive exploration to discover successful trajectories.
- **Learning improves sharply after the first reward**. Once one correct sequence is identified, Q-values rapidly increase around relevant states.

In conclusion, the results demonstrate the advantages of SARSA(λ): faster credit propagation, improved sample efficiency, and more stable learning compared with one-step TD methods. Despite the intrinsic stochasticity of Taxi-v3, the agent consistently converged to a functional policy, confirming the effectiveness of eligibility traces in environments with sparse and delayed rewards.

2.2 Mountain Car: Value function approximation via Q-learning TD(λ) with RBF

The goal of this exercise is to implement an agent based on Temporal Difference learning with eligibility traces, in this case TD(λ), combinated with a Linear Value Function Approximator. The environment contains a continuous state space and a discrete action set. Since tabular methods cannot be applied, the agent must rely on a feature representation to approximate the action–value function:

$$Q(s,a) \approx w_a^\top \phi(s), \tag{3}$$

where $\phi(s)$ is a vector of nonlinear features obtained through Radial Basis Function (RBF).

2.2.1 Description of Q-learning TD(λ)

The main aim is to learn a Q-function $Q(s,a)$ that tell how good each action is in a given state, in terms of expected future rewards.

- **TD(λ) Concept:** Combines Temporal Difference learning, which updates estimates during the episode, with eligibility traces, a memory of how recently each state-action pair was visited.
  - TD(0): updates only the current state–action.
  - TD(1): updates all past state–actions like Monte Carlo.
  - TD(λ): interpolates between TD(0) and TD(1) using trace decay parameter λ.
- **Eligibility Traces:** Maintain a memory $e(s,a)$ of recently visited state–action pairs, update each trace: $e(s,a) \leftarrow \gamma\lambda e(s,a)$ and boost the current trace: $e(s_t,a_t) \leftarrow e(s_t,a_t) + 1$.
- **Temporal Difference Error:** the update is based on the Temporal Difference error:

  $$\delta_t = r_t + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t)$$

  This measures the difference between what we expected and what actually happened.
- **Weight Update (Backward View):** For all actions:

  $$Q(s,a) \leftarrow Q(s,a) + \alpha\, \delta_t\, e(s,a)$$

  where α is the learning rate, λ the eligibility decay, and γ the discount factor.
- **Exploration:** Use ϵ-greedy policy: select a random action with probability ϵ, otherwise $\arg\max_a Q(s,a)$.
- **Advantages:**
  - Assigns credit efficiently to multiple past state–actions.
  - Combines short-term updates with long-term credit assignment.
  - Works with function approximation for continuous states.

2.2.2 Implementation

In this assignment, we modified two main parts. First, in the `RBFFeatureEncoder` class, we added state normalization, created RBF features at multiple scales, and implemented the `encode()` and `size` methods to produce a fixed-length feature vector for each state. Second, in the `TDLambda_LVFA` class, we implemented the TD(λ) update in `update_transition()`, computing the TD error, updating eligibility traces, and adjusting the weights accordingly.

In the `RBFFeatureEncoder` class the code generates 10 000 samples from the environment's observation space. These samples are used to fit a `StandardScaler`, ensuring that each dimension of the state space has zero mean and unit variance:

$$\hat{s} = \text{StandardScaler}(s).$$

Then the code required the construction of an explicit RBF feature map via `RBFSampler`. Since a single RBF kernel is often insufficient to capture the complexity of the environment, I used a multi-scale feature representation by combining several RBF layers with different kernel widths (γ values):

- wide kernels ($\gamma = 1.0$) capture coarse structure,
- medium kernels ($\gamma = 2.0$),
- narrow kernels ($\gamma = 5.0$) capture fine variations.

These layers are concatenated using `FeatureUnion`. Each sampler produces a set of random Fourier features approximating the RBF kernel:

$$\phi(s) = \big[\phi_{\gamma_1}(s),\ \phi_{\gamma_2}(s),\ \phi_{\gamma_3}(s)\big].$$

Then the `encode` function applies scaling and then transforms the state using the full RBF feature mapping. The returned vector is the feature representation used by the linear approximator.

At the end, the `size` property, returns the total number of features produced by the combined RBF encoders. This determines the dimensionality of the weight matrix.

The final feature vector has size

$$200 + 200 + 300 = 700,$$

leading to a weight matrix of dimension

$$W \in \mathbb{R}^{|\mathcal{A}| \times 700}.$$

The core of the learning algorithm is the TD(λ) update rule with replacing eligibility traces in the `def update_transition(self, s, action, s_prime, reward, done)`.

Given state s, action a, next state s', reward r, and features $\phi(s)$ and $\phi(s')$, the update computes:

$$\delta_t = r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

Eligibility traces are updated as:

$$e_a \leftarrow \gamma\lambda e_a + \phi(s)$$

Finally, weights are updated only for the chosen action:

$$W_a \leftarrow W_a + \alpha\, \delta_t\, e_a.$$

This implements online backward-view TD(λ) for a linear action-value function:

$$Q(s, a) = W_a^\top \phi(s).$$

2.2.3 Analysis of Training Results and Evaluation Results

The training process runs for 200 episodes, with:

- decaying ϵ for exploration,
- optional α decay,
- evaluation every 20 episodes.

Initial performance is around -200, as expected for a random or untrained policy. Over time, the evaluation score improves and fluctuates between approximately -120 and -130 depending on the episode.

After training, the agent was evaluated multiple times (15+ runs). The reported *Mean Reward* from different evaluation runs ranged between:

$$-106.4 \quad \text{and} \quad -140.5.$$

This variability is expected due to:

- stochastic initial states in MountainCar (the environment is not deterministic),
- the inherent randomness of the policy due to the learned value landscape,
- the sensitivity of linear function approximation with high-dimensional RBF features.

On average, the agent consistently achieved performance between -115 and -130, which represents a substantial improvement over the initial baseline around -200.

Discussion and Conclusions The RBF feature encoder significantly improves the agent's ability to model the continuous state space compared to raw features. Multi-scale RBF kernels provide local and global sensitivity to different regions of the state manifold.

The variability in performance indicates that although the representation is strong, the agent does not converge to a fully stable policy. This is common in MountainCar when using linear approximators: the problem is sparse, requires coordinated long-term planning, and TD(λ) can propagate the reward only slowly back through the feature space.

Overall, the results demonstrate:

- stable learning dynamics,
- smooth (though noisy) improvement over training episodes,
- meaningful generalization using the RBF encoder,
- but also non-negligible variance typical of MountainCar with LVFA.

3 Collaborations

I discussed the assignment with Flaminia Lanzellotti, matricola number 2201485.